

PROJECT DOCUMENTATION

RetailMart UI Automation Framework

End-to-End UI Test Automation using Playwright & TypeScript

GitHub Link: “https://github.com/RajaVUst/SDET-3/tree/Saiteja_Final_Assessment”

Field	Details
Application Under Test	RetailMart E-Commerce Web Application
Automation Tool	Playwright
Programming Language	TypeScript
Framework Design	Page Object Model (POM)
Prepared By	SDET Automation Engineer
Document Type	Assessment Submission

Table of Contents

Table of Contents	2
1. Project Overview	3
1.1 Objective	3
1.2 Project Details	3
1.3 Technology Stack	3
2. Folder Structure	4
3. Framework Architecture	5
4. Environment Configuration	5
5. Implemented Test Scenarios	6
5.1 Scenario 1: Free Shipping Threshold Validation	6
5.2 Scenario 2: Payment Processing Error Validation	7
6. CI/CD — GitHub Actions	9
6.1 Workflow Steps	9
7. Execution Steps	Error! Bookmark not defined.
8. Deliverables	11
9. Challenges Faced	11
10. Future Enhancements	Error! Bookmark not defined.
11. Conclusion	Error! Bookmark not defined.
Appendix A: Screenshot Index	Error! Bookmark not defined.

1. Project Overview

1.1 Objective

This project automates the RetailMart web application using Playwright with a reusable Page Object Model (POM) framework, following industry best practices for scalable and maintainable UI test automation.

1.2 Project Details

Attribute	Value
Project Name	RetailMart UI Automation Framework
Application	RetailMart E-Commerce Web Application
Application URL	https://chess-agent-83252463.figma.site/
Automation Tool	Playwright
Programming Language	TypeScript
Framework Design	Page Object Model (POM)

1.3 Technology Stack

Technology	Purpose
Node.js	JavaScript runtime for executing the framework
Playwright	Browser automation and test execution engine
TypeScript	Strongly typed scripting language for maintainable code
Git	Version control for source code management
GitHub Actions	CI/CD pipeline for automated test execution
dotenv	Environment variable management
Playwright HTML Report	Test execution reporting and analysis

2. Folder Structure

The framework follows a modular and layered folder structure that separates concerns such as pages, locators, flows, fixtures, and test data. This improves readability, reusability, and long-term maintainability.

```
RetailMart-UI-Automation/  
|-- .github/  
|   |-- workflows/  
|       |-- playwright.yml  
|-- src/  
|   |-- pages/  
|   |   |-- HomePage.ts  
|   |   |-- SearchResultsPage.ts  
|   |   |-- ProductPage.ts  
|   |   |-- CartPage.ts  
|   |   |-- CheckoutPage.ts  
|   |   |-- PaymentPage.ts  
|   |-- locators/  
|   |   |-- (element locators per page)  
|   |-- flows/  
|   |   |-- ShippingFlow.ts  
|   |   |-- PaymentFlow.ts  
|   |-- fixtures/  
|   |   |-- baseTest.ts  
|   |-- data/  
|   |   |-- testData.ts  
|   |-- utils/  
|   |   |-- Logger.ts  
|   |-- config/  
|       |-- Environment.ts  
|-- tests/  
|   |-- freeShipping.spec.ts  
|   |-- paymentError.spec.ts  
|-- playwright-report/  
|-- test-results/  
|-- .env  
|-- playwright.config.ts  
|-- package.json  
|-- tsconfig.json
```

3. Framework Architecture

The framework is built on a layered architecture, keeping test logic, page interactions, and data completely separate. Each layer has a single, well-defined responsibility, as described below.

Component	Purpose
Pages	Represent individual web pages of the application and expose page-specific methods
Locators	Store element selectors separately, keeping pages clean and easy to maintain
Flows	Combine multiple page actions into reusable, business-level workflows (e.g. checkout)
Fixtures	Provide shared setup such as browser context, page instances, and test hooks
Test Data	Centralized static and dynamic data used across test scenarios
Logger	Captures structured execution logs for debugging and traceability
Environment	Loads configuration such as Base URL and credentials from the .env file
Tests	Contain test scenarios that assert expected application behavior

4. Environment Configuration

Configuration values such as the Base URL and payment card details are maintained in a .env file and are never hardcoded in the test scripts. These values are loaded at runtime through Environment.ts using the dotenv package.

Sample .env file

```
BASE_URL=https://chess-agent-83252463.figma.site/  
VALID_CARD_NUMBER=4111111111111111  
INVALID_CARD_NUMBER=4000000000000002  
CUSTOMER_NAME=John Doe  
CUSTOMER_EMAIL=john.doe@example.com
```

Environment.ts (Concept)

Environment.ts reads variables from .env using dotenv and exposes them as strongly typed constants that are imported wherever configuration values are required, such as inside Page classes, Flows, and Playwright configuration.

5. Implemented Test Scenarios

5.1 Scenario 1: Free Shipping Threshold Validation

Objective

Verify that shipping charges are updated correctly when the free shipping threshold is reached.

Test Flow

- Search for the product 'QuickCharge'
- Add the product to the cart
- Open the Shopping Cart
- Verify the message 'Add \$X more for free shipping'
- Increase the product quantity
- Verify shipping charge changes to FREE
- Verify the order summary is updated correctly

Before Free Shipping:

Shopping Cart (1 item)

Add \$40.01 more for **free shipping!**



QuickCharge 65W USB-C Hub

QuickCharge

- 1 +

Remove

\$34.99

[← Continue Shopping](#)

Order Summary

Subtotal (1 items)	\$34.99
Tax (8.5%)	\$2.97
Shipping	\$5.99
Total	\$43.95

[Proceed to Checkout](#)

Secure checkout — demo only, no real charges

After Free Shipping:

Shopping Cart (3 items)

✓ You qualify for **free shipping!**



QuickCharge 65W USB-C Hub

QuickCharge

- 3 +

Remove

\$104.97

\$34.99 each

[← Continue Shopping](#)

Order Summary

Subtotal (3 items)	\$104.97
Tax (8.5%)	\$8.92
Shipping	FREE
Total	\$113.89

[Proceed to Checkout](#)

Secure checkout — demo only, no real charges

5.2 Scenario 2: Payment Processing Error Validation

Objective

Verify the application's behavior when a payment attempt fails, and confirm successful order placement on retry with valid payment details.

Test Flow

- Search for the product
- Add the product to the cart
- Proceed to Checkout
- Enter customer information
- Continue to the Payment step
- Enter an invalid card number
- Click 'Place Order'
- Verify the payment processing error message
- Retry payment using a valid card number
- Verify the Order Confirmation page is displayed

Payment Processing Error:

 Payment processing error. Please check your card details and try again.



Secure Payment

Demo — no real charge

Cardholder Name *

saiteja

Card Number *

4111 1111 1111 1111

Expiry (MM/YY) *

12/27

CVV *

123

Place Order — \$113.89

Successfully Order Placed:**Order Confirmed!**

Thank you, **saiteja**! Your order has been placed successfully.

Your Order Number

RM-20260721-68989

Placed on July 21, 2026

Processing

Items Ordered

QuickCharge 65W USB-C Hub

Qty: 3

\$104.97

6. CI/CD — GitHub Actions

The framework is integrated with GitHub Actions to automatically execute the test suite on every code change, ensuring continuous validation of application quality without manual intervention.

6.1 Workflow Steps

Step	Description
Checkout Repository	Pulls the latest source code from the repository
Setup Node.js	Configures the required Node.js runtime version
Install Dependencies	Installs all npm packages defined in package.json
Install Playwright Browsers	Downloads browser binaries required for test execution
Execute Tests	Runs the Playwright test suite (npx playwright test)
Upload Playwright HTML Report	Publishes the HTML report as a workflow artifact
Upload Test Results	Publishes raw test results/traces as a workflow artifact

GitHub WorkFlow:

The screenshot shows a GitHub Actions workflow run for the repository 'Playwright Automation'. The workflow is named 'Submitting Final Assessment #2' and has a green checkmark indicating it was successful. It was triggered by a push 18 minutes ago from user 'Saitejakodi' with commit 'ac99e5c'. The workflow file is 'ci.yml' and it runs on the 'push' event. The workflow consists of a single step named 'playwright-tests' which took 54s to complete. The status is 'Success'. The total duration of the workflow is 57s and there is 1 artifact. The left sidebar shows the workflow file 'ci.yml' and the step 'playwright-tests'. The right sidebar shows the 'Annotations' section with 2 warnings, one of which is 'playwright-tests' with a warning that 'Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checko...'. A 'Re-run all jobs' button is visible in the top right corner.

7. Execution Steps

The commands below outline how to set up and run the automation framework locally.

```
# Install project dependencies
npm install

# Install Playwright browsers
npx playwright install

# Execute the test suite
npx playwright test

# View the HTML test report
npx playwright show-report
```

8. Deliverables

Upon execution, the framework produces the following outputs:

- Playwright HTML Report — a detailed, interactive report of test execution results
- Test Results — raw output including traces, screenshots, and logs for failed tests
- GitHub Actions Pipeline — an automated CI/CD pipeline that runs on every commit or pull request

9. Challenges Faced

- Handling dynamic web elements that load asynchronously or change state during test execution
- Designing a reusable and maintainable framework structure that scales with new test cases
- Managing environment variables securely across local and CI/CD execution environments
- Validating the payment retry flow after an intentional payment failure
- Accurately validating the free shipping threshold logic and dynamic order summary updates

10. Conclusion

The RetailMart UI Automation Framework demonstrates the successful implementation of a scalable Playwright automation solution using the Page Object Model design pattern. By separating pages, locators, flows, fixtures, and test data into distinct layers, the framework achieves high reusability and maintainability.

Integration with GitHub Actions further extends the framework's value by enabling continuous, automated validation of the application on every code change, along with easy access to detailed HTML reports. Overall, this project reflects industry-standard automation engineering practices suitable for real-world QA delivery.